Healthcare Cloud Access Control

Current Project Documentation & Commands

MSc Cyber Security Project

Project title: *Access Controls for Cloud Data: Implementation & Analysis*

Current status: Local Python RBAC prototype implemented; 19 pytest tests passed in the user's reported run. AWS integration has not started.

Environment: macOS, Python 3.15.2, VS Code, pytest 9.2.0. Uses synthetic test identities and record metadata only.

This document records the current learning prototype. It is not production-ready, and the local test-user registry does not provide real authentication.

1. Current Project Structure

```
healthcare-cloud-security/
■■■ .venv/
■■■ src/
■   ■■■ access_control.py
■   ■■■ healthcare_data.py
■■■ tests/
■   ■■■ test_access_control.py
■■■ docs/
■■■ results/
```

File responsibilities:

• **src/healthcare_data.py** — synthetic test users and record ownership metadata.

• **src/access_control.py** — trusted test-user lookup, RBAC permissions and patient record ownership checks.

• **tests/test_access_control.py** — 19 automated tests.

• **.venv/** — local Python virtual environment.

2. Commands to Run the Project

Open the project folder

```
cd ~/Sreelakshmi/Project/healthcare-cloud-security
```

This path matches the terminal output supplied for the current project. If you move the folder, use its new path.

Activate the virtual environment

```
source .venv/bin/activate
```

The terminal prompt should show (.venv) when the environment is active.

Check Python and pytest

```
python --version
python -m pytest --version
```

Run the full test suite

```
python -m pytest -v
```

Your reported successful run collected 19 items and finished with 19 passed in 0.01s. Re-run tests after code changes.

Run only the access-control tests

```
python -m pytest -v tests/test_access_control.py
```

Run tests with a concise summary

```
python -m pytest -q
```

Deactivate the virtual environment

```
deactivate
```

Reactivate later with: source .venv/bin/activate

3. Current Source Code

File 1 — src/healthcare_data.py

```
"""
Synthetic test users and healthcare record metadata.
Learning prototype only. No real patient data.
"""

TEST_USERS = {
    "doctor_001": {
        "role": "doctor",
        "patient_id": None,
    },
    "nurse_001": {
        "role": "nurse",
        "patient_id": None,
    },
    "admin_001": {
```

```
        "role": "admin",
        "patient_id": None,
    },
    "patient_alice": {
        "role": "patient",
        "patient_id": "patient_001",
    },
    "patient_bob": {
        "role": "patient",
        "patient_id": "patient_002",
    },
}

SYNTHETIC_RECORDS = {
    "record_001": {
        "owner_patient_id": "patient_001",
    },
    "record_002": {
        "owner_patient_id": "patient_002",
    },
}
```

File 2 — src/access_control.py

```
"""
Healthcare Cloud Access Control
Step 3: Trusted test-user lookup, RBAC and ownership checks.

Learning prototype only.
Uses synthetic test data and does not provide real authentication.
"""

from healthcare_data import TEST_USERS, SYNTHETIC_RECORDS


ROLE_PERMISSIONS = {
    "doctor": {"read", "update"},
    "nurse": {"read"},
    "admin": {"read", "upload", "delete"},
    "patient": {"read"},
}


def get_trusted_user(user_id: str):
    """
    Return a registered test user's details.
    Unknown or invalid user IDs return None.
    """
    if not isinstance(user_id, str):
        return None

    user_id = user_id.strip()

    if not user_id:
        return None

    return TEST_USERS.get(user_id)


def check_access(user_id: str, action: str, record_id: str) -> bool:
    """
    Check whether a registered test user may perform an action
    on a synthetic healthcare record.
    """
    if not all(
        isinstance(value, str)
        for value in (user_id, action, record_id)
    ):
        return False

    user_id = user_id.strip()
    action = action.strip().lower()
    record_id = record_id.strip()

    if not user_id or not action or not record_id:
        return False

    user = get_trusted_user(user_id)

    if user is None:
        return False

    role = user["role"]
    record = SYNTHETIC_RECORDS.get(record_id)

    if record is None:
        return False
```

```
    allowed_actions = ROLE_PERMISSIONS.get(role, set())

    if action not in allowed_actions:
        return False

    if role == "patient":
        if action != "read":
            return False

        if user["patient_id"] != record["owner_patient_id"]:
            return False

    return True
```

File 3 — tests/test_access_control.py

```
import sys
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT / "src"))

from access_control import check_access, get_trusted_user


# Trusted-user lookup tests
def test_registered_user_is_found():
    user = get_trusted_user("patient_alice")
    assert user is not None
    assert user["role"] == "patient"
    assert user["patient_id"] == "patient_001"


def test_unknown_user_is_rejected():
    assert get_trusted_user("unknown_user") is None


def test_invalid_user_id_is_rejected():
    assert get_trusted_user(None) is None


def test_whitespace_user_id_is_rejected():
    assert get_trusted_user("   ") is None


# RBAC permission tests
def test_doctor_can_read_record():
    assert check_access("doctor_001", "read", "record_001") is True


def test_doctor_can_update_record():
    assert check_access("doctor_001", "update", "record_001") is True


def test_nurse_cannot_delete_record():
    assert check_access("nurse_001", "delete", "record_001") is False


def test_patient_cannot_update_record():
    assert check_access("patient_alice", "update", "record_001") is False


def test_unknown_user_cannot_access_record():
    assert check_access("unknown_user", "read", "record_001") is False


def test_unknown_action_is_denied():
    assert check_access("doctor_001", "download_all", "record_001") is False


# Patient ownership tests
def test_patient_can_read_own_record():
    assert check_access("patient_alice", "read", "record_001") is True


def test_patient_cannot_read_another_patients_record():
    assert check_access("patient_alice", "read", "record_002") is False


def test_second_patient_can_read_own_record():
    assert check_access("patient_bob", "read", "record_002") is True


# Missing record and invalid input tests
def test_missing_record_is_denied():
    assert check_access("doctor_001", "read", "record_999") is False

def test_empty_record_id_is_denied():
```

```
    assert check_access("doctor_001", "read", "") is False


def test_none_user_id_is_denied():
    assert check_access(None, "read", "record_001") is False


def test_none_action_is_denied():
    assert check_access("doctor_001", None, "record_001") is False


def test_none_record_id_is_denied():
    assert check_access("doctor_001", "read", None) is False


def test_action_is_case_insensitive():
    assert check_access("doctor_001", "READ", "record_001") is True
```

4. Manual Demonstration Command

```
python -c "import sys; sys.path.insert(0, 'src'); from access_control import check_access; print('Alice reads Alice:', check_access('p
```

Expected output:

```
Alice reads Alice: True
Alice reads Bob: False
Nurse deletes record: False
Unknown user: False
Doctor updates record: True
```

5. Test Results

Test category	Tests	What it checks
Trusted-user lookup	4	Registered, unknown, invalid and blank user IDs
RBAC permissions	6	Permitted and prohibited actions
Patient ownership	3	Own-record access versus another patient's record
Invalid and missing input	5	Missing records, invalid values and None inputs
Case-insensitive action	1	Uppercase action input
Total	**19**	**All passed in the reported run**

These results provide initial evidence for the tested cases only. They do not prove that every possible attack has been prevented.

6. Draft MSc Report Text

Initial RBAC Implementation and Testing

A local Python prototype was developed to investigate Role-Based Access Control (RBAC) within a simulated healthcare environment. The prototype uses synthetic user identities and healthcare record metadata to evaluate access decisions without processing real patient information.

The implementation includes registered-user lookup, role-based permission checks, and patient record ownership enforcement. A pytest test suite was created to evaluate authorised and unauthorised actions, unknown users, invalid inputs, missing records, and attempts by patients to access records belonging to other patients.

The initial test execution completed successfully, with **19 tests passed and no failures**. These results provide initial evidence that the implemented access-control rules behave as expected for the tested scenarios. However, the test suite does not demonstrate resistance to every possible attack, and the local user registry does not provide real authentication.

Further work will strengthen the test coverage, compare RBAC with Attribute-Based Access Control (ABAC), and evaluate access-control effectiveness using defined security test cases. Cloud integration will be considered only after the local design and evaluation are sufficiently developed.

7. Limitations and Security Notes

• The user registry is simulated and does not authenticate a person or verify a login.

• A caller who can supply another registered user's ID could impersonate that identity.

• Doctor permissions are broad and do not yet model care relationships or finer-grained clinical rules.

• The prototype checks access decisions but does not actually read, update, upload or delete record contents.

• AWS S3, IAM policies, encryption, logging and cloud security tests have not been implemented.

• Use synthetic data only. Do not add real patient records, passwords, AWS keys or other secrets.

8. Suggested Next Stages

• Review security gaps and strengthen tests.

- Implement an ABAC prototype and compare it with RBAC.
- Define measurable evaluation criteria and record test results.
- Design AWS S3 and IAM integration.
- Integrate cloud storage only after reviewing the local security design.
- Analyse findings and write the final MSc report.

Confirm the final project scope, permission model and evaluation requirements with your supervisor.